CMDBUF RD 仿真基线与历史记录

早期设计 仿真计划 注错计划与阶段报告

文档版本 2026-10-13　代码基线 1949e75

本文用于设计评审、RTL 集成、验证复核和实际 GUI 操作。项目说明以 PDF 与 Word 交付：PDF 用于阅读与打印，Word 保留可编辑的正文、表格和图片。

当前芯片 RTL 为 Verilog-2001。SBY 已实际完成完整求解；当前设备未安装真实 FormalMC，模型交叉验证与能力入口不能当作 FormalMC 实机证明。

历史设计建议、阶段报告和截图均保留所属版本。正文中的历史 pending、工具缺失或单任务结果，不改写为当前完整验证 PASS。

阅读方式

按主题阅读下列分册，或使用 PDF 书签和 Word 导航窗格。时序图采用横向页面。GUI 截图保持真实像素，并提供关键操作区域的放大视图；放大仅改变文档中的图片视口。

01 项目使用与部署说明

02 CMDBUF RD 详细设计方案

03 CMDBUF RD 时序图集

04 CMDBUF RD 注错与容量保护清单

05 UCAgent Tk GUI 详细操作说明

06 CMDBUF RD Formal 验证说明

07 CMDBUF RD 仿真基线与历史记录

现有 Markdown 和 HTML 是可追溯的内容源与历史阅读页，不是新增说明的最终交付格式。代码、约束、机器可读结果和工具自带第三方文档保持各自原格式。

CMDBUF RD 仿真基线与历史记录

本分册完整保留早期仿真阶段的说明和设计基线，用于追溯。其 SystemVerilog 文件名、尚未完成的验证项、旧路径或阶段结论均属于当时版本；当前 Verilog 实现和正式验证结果见分册 02 与 06。

内容来源　simulation/cmdbuf_rd/README.md

cmdbuf_rd RTL 与验证工程

当前芯片 RTL 已转换为 Verilog-2001，`rtl/*.v` 为集成文件；新的等价性与完整重新验证记录见 Verilog 迁移说明。以下早期回归与注错报告保留原执行时间。

开发与实际编译、仿真、注错验证主机：192.168.31.116，工程：`/root/cmdbuf_rd`。 本工程依据已对齐的 cmdbuf_rd 设计基线实现。以下内容描述早期功能验证；完整形式证明及本次语言转换的当前结果见仓库验证报告。综合面积和静态时序仍未签核。

实现结构

- `rtl/cmdbuf_rd.v`：顶层接口、CMDBUF/OTB 连接。
- `rtl/cmdbuf_rd_cmd.v`：9 项元数据与状态、年龄矩阵、最老 5 条待发窗口、三周期接收/判断/发射、5A/6B、容量准入、每 XAU 保序及响应 RR、10B 同拍复用。
- `rtl/cmdbuf_rd_otb.v`：9 项 128-bit 数据与 pc_err、成功发射四拍返回标记、无返回背压的数据捕获、分配/退休数据清除。
- `/root/env_base/adapters/cmdbuf_rd/`：独立 C++ 参考模型和 Verilator 驱动。
- `/root/env_base/cases/cmdbuf_rd/`：定向及随机 case；公共 runner 未加入 DUT 特例。
- `scripts/run_mutations.py`：隔离 RTL 注错副本、编译与检测结果分类。
- `docs/`：设计来源、实现选择、用例及注错说明。
- `results/`：检查结果、逐用例 JSON/日志、需要保留的波形。

开发基线

AR 和 R 均共享负载，每周期最多接收、发射、捕获、返回各一条。arid 是透传标签，顺序由内部接收年龄决定。 `rd_mem_status[1:0][7:0]` 的第一个索引为奇偶 half，第二个索引为 bank；0 表示空闲。

Arbit 地址为 12-bit `araddr[17:6]`。 CMDBUF 与 OTB 的 9 项一一对应，从合法 AR 握手起保留到实际 R 返回。

保留容量采用“每 XAU 1 项逻辑保底 + 6 项共享”，物理 entry 仍统一分配。占用包含所有已接收未返回命令；`sum(max(occ[x],1)) <= 9`。这是满足用户禁止容量独占要求的实现选择。上游仲裁公平性未确定，因此只保证容量不会被其他来源侵占，不能保证上游一定调度某个来源。

选择先构造成功前馈后的最老 5 条未成功发射命令，再按 Bank 空闲过滤并选最老；当前请求级命令不可重复选中。5A 失败不冲刷下一条预选；6B 成功当拍推进窗口；10B 返回当拍允许新接收复用。 成功取同一 req bank 上 `ar_rd_gnt` 和 `rw_rd_gnt` 同为 1。成功 C3 → MEM 捕获 C7 → 最早 R 返回 C8。 地址越界仍走 MEM，R 状态优先级为地址越界 11、奇偶异常 01、正常 00。R 仅在 ready 的已完成来源队头之间 RR。

已采用但需要系统集成确认的实现选择

1. 异步低有效 `rst_n`；所有状态和数据复位清零。运行中复位的 MEM 协同协议不包含在本次接口定义中，集成时仍需确认。
2. RR 顺序 0→1→2，复位起点 0，仅实际响应后更新为获胜来源的下一来源，空闲保持。
3. 错误响应的 rdata 仍透传已捕获的 MEM 数据，rresp 指示错误。
4. 选择进入请求级后，下一周期仍发送 req；Bank 状态变化由当拍双 grant 最终决定成功。
5. 无有效请求/响应时，输出负载置零。OTB 分配、退休和复位均物理清零数据。
6. src=3 不接受；已接受请求的 arresp 只允许 00/11。其他编码属于未定义输入，内部断言会报错。
7. 所有接口均同一时钟域；没有 CDC、ECC、超时或可变返回延迟处理。MEM 必须严格成功后四拍给出有效数据。

复现

在虚拟机中执行：

```
cd /root/cmdbuf_rd
make lint validate
make test
python3 scripts/run_mutations.py --help
```

`make test` 调用既有公共 runner；使用 `/opt/rh/devtoolset-11/enable` 中的 GCC 11，Verilator 5.024，C++17。不要使用系统默认 GCC 4.8 编译 harness。 常规结果在 `results/baseline/regression.json`；单个 case 可通过公共 Makefile 的 `run-case CASE=<绝对路径>` 运行。补充检查和注错的实际命令、统计以 `docs/verification_baseline.md`、`docs/mutation_plan.md` 及最终验证报告为准。

验证边界

独立参考模型按接收序号维护队列、返回日历和每来源队头，逐拍核对 req、地址、ready、pending 与完整 R 负载。RTL 内部断言额外检查容量、年龄和生命周期一致性。 注错检测区分参考模型、RTL 断言、构建失败、运行环境错误及漏检。既有 49 组注错清单是规格覆盖来源，不表示已经实际运行 49 种 RTL 变异。 测试通过不代表所有输入序列的形式证明，也不代表目标频率已经收敛。

本目录不保存 SSH 密码；本机源文件、结果及随交付包提供的 SHA256 用于追溯实际验证版本。

内容来源　simulation/cmdbuf_rd/docs/verification_baseline.md

cmdbuf_rd 仿真验证基线报告

日期：2026-10-10。执行目录为 Linux VM 的 `/root/cmdbuf_rd`；本报告由实际 case JSON 汇总。RTL 的 Bank half 索引等价改写后已重新构建并重跑。

结果

Verilator 5.024、C++17、`--Wall --assert --trace`：基础 19/19 PASS，补充 4/4 PASS；各 case 的必达覆盖计数均满足要求。未关闭 lint 警告或 RTL 断言。

集合	case	功能周期	比较/检查	接收	成功发射	MEM 捕获	R 退休
baseline	19	6433	101329	3126	3126	3126	3126
extended	4	300	3658	52	52	52	52
combined	23	6733	104987	3178	3178	3178	3178

功能周期包含正常刺激、排空和末尾空闲检查，不包含复位拉低期间的时钟。每次检查是参考模型比较或明确断言意图的一次判定，不等价于 formal 属性覆盖数。

可复现命令

基础回归：

```
make -f /root/env_base/env/Makefile PROJECT_CONFIG=/root/env_base/projects/cmdbuf_rd.mk run-all
```

补充回归使用独立 adapter、build 和结果目录，不改变基础 harness：

```
make -f /root/env_base/env/Makefile \
  PROJECT_CONFIG=/root/env_base/projects/cmdbuf_rd.mk \
  HARNESS_FILES=/root/env_base/adapters/cmdbuf_rd/cmdbuf_rd_harness_extended.cpp \
  CASE_DIR=/root/env_base/cases/cmdbuf_rd/extended \
  BUILD_DIR=/root/cmdbuf_rd/build/verilator_extended \
  RESULT_DIR=/root/cmdbuf_rd/results/extended run-all
```

用例明细

Case	结果	周期	检查	接收/退休
01_pipeline	PASS	51	793	24/24
02_same_xau_ooo	PASS	43	514	6/6
03_forward_6b	PASS	43	513	6/6
04_grants	PASS	103	1333	24/24
05_failure_5a	PASS	43	537	8/8
06_window_block	PASS	48	563	6/6
07_single_pulse	PASS	48	575	7/7
08_rr	PASS	53	648	9/9
09_errors	PASS	38	443	4/4
10_capacity_reuse	PASS	53	669	11/11
11_capacity_700	PASS	53	648	9/9
12_capacity_440	PASS	53	647	9/9
13_tokens	PASS	63	737	8/8

Case	结果	周期	检查	接收/退休
14_single_source_throughput	PASS	170	3478	160/160
15_reset_clean	PASS	96	1256	24/24
16_random_seed_1	PASS	1814	29188	935/935
17_random_seed_17	PASS	1812	29234	942/942
18_random_seed_91	PASS	1816	29172	931/931
19_rr_ready_masks	PASS	33	381	3/3
20_all_bank_halves	PASS	163	1839	16/16
21_repeated_ids	PASS	58	697	9/9
22_status_after_selection	PASS	28	331	3/3
23_nonmonotonic_ids	PASS	51	791	24/24

独立参考模型与检查范围

- 参考模型按实际 AR 接收顺序维护命令列表，不照搬 RTL 年龄矩阵或物理 entry 分配。
- 每周期独立计算原队头、RR、当拍退休后的逐源 ready、成功前馈后的最老五项窗口、下一拍请求。
- 每次成功发射建立 +4 拍返回日历；MEM 数据仅在对应周期有效，无效拍驱动不同噪声数据。R 候选只读取此前已捕获的完成状态。
- 比较 req、完整 12-bit 下游地址、pending_cnt、三位 aready/rvalid 及共享 R 的全部 128-bit 数据与元数据。
- 失败用例打印 CHECK_FAILED、返回非零并保留 VCD；配置错误为 CONFIG_ERROR/退出 2，不能作为成功检出 RTL 注错。

定向覆盖审查

- 复位：所有用例检查初始低有效异步复位输出；15_reset_clean 额外排空后重新复位并再发送一批请求，两个批次共 24 条。未将运行中仍有未完成 MEM 返回的复位冲刷行为擅自定义为已支持。
- 所有 16 个 Bank half：20_all_bank_halves 每次只放开目标 half，其余 15 个 half 均忙，8 Bank × 2 half 全部成功发射并比对返回。
- ID：21_repeated_ids 使用同一 arid=0x5a，同时存在同源重复 ID，通过不同地址、port_id 和数据确认归属；23_nonmonotonic_ids 显式命中非单调 ID。
- 选择后 status 翻转：22_status_after_selection 中两条请求在判断拍空闲、请求拍变忙，但双 grant 有效；仍正确发射，不二次撤销。
- 单源吞吐：14_single_source_throughput 连续接收 160 条，159 次相邻拍连续接收；每条均满足最短三拍前向路径，153 次七项占用下退休/接收并行和四事件并发。
- 容量：明确覆盖 (7,0,0) ready=110、(4,4,0) ready=100、满表来源唯一项退休 ready=010、共享项退休 ready=111，以及两次满表同拍复用。
- 错误：arresp=00/11 与 pc_err=0/1 四组合均返回正确优先级；零数据且 pc_err=0 的单拍返回单独检查。

以下为基础回归累计事件计数；每个定向 case 同时有独立最小门槛，不能以其他 case 的覆盖替代：

覆盖事件	次数
three_cycle_path	449
failed_keep_next	2117
retry_success	1280
sixth_success_forward	2
outside_window_blocked	14
ar_only_grant	918
rw_only_grant	703
wrong_bank_grant	408
capture_without_ready	404
zero_data_no_error	149
capture_after_bubble	1355
same_xau_ooo_issue	446
same_xau_head_block	1062
rr_all_three	45
rr_skip	1892
capacity700_ready110	22
capacity440_ready100	18
rejected_unready_src	150
unique_retire_ready010	2

覆盖事件	次数
shared_retire_ready111	6
full_reuse	1442
single_source_turnover	181
four_events	745
error_combo_00	2476
error_combo_01	370
error_combo_10	244
error_combo_11	36
drained_reset	1

补充回归的明确覆盖：

覆盖事件	次数
duplicate_live_id	6
issued_half_0_bank_0	7
issued_half_0_bank_1	1
issued_half_0_bank_2	6
issued_half_0_bank_3	1
issued_half_0_bank_4	5
issued_half_0_bank_5	1
issued_half_0_bank_6	5

覆盖事件	次数
issued_half_0_bank_7	1
issued_half_1_bank_0	1
issued_half_1_bank_1	6
issued_half_1_bank_2	1
issued_half_1_bank_3	5
issued_half_1_bank_4	1
issued_half_1_bank_5	5
issued_half_1_bank_6	1
issued_half_1_bank_7	5
nonmonotonic_live_id	21
request_busy_but_granted	2

注：上表 half 总数为四个补充 case 合计；20_all_bank_halves 自身对每个 half 都设置且通过至少一次的门槛。

产物

- `/root/env_base/adapters/cmdbuf_rd/cmdbuf_rd_harness.cpp`：基础参考模型。
- `/root/env_base/adapters/cmdbuf_rd/cmdbuf_rd_harness_extended.cpp`：补充定向刺激与覆盖版本，原模型算法保持一致。
- `/root/env_base/cases/cmdbuf_rd/*.case`：基础 19 用例。
- `/root/env_base/cases/cmdbuf_rd/extended/*.case`：补充 4 用例。
- `/root/cmdbuf_rd/results/baseline/` 与 `/root/cmdbuf_rd/results/extended/`：逐 case JSON、日志、runner summary。

- `/root/cmdbuf_rd/results/baseline/05_failure_5a.vcd、10_capacity_reuse.vcd、19_rr_ready_masks.vcd`：指定保留波形。
- `/root/cmdbuf_rd/results/extended/22_status_after_selection.vcd`：选择后 Bank 状态翻转波形。
- `/root/cmdbuf_rd/docs/verification_summary.json`：本报告同源的结构化汇总。

这是固定参数配置的动态仿真基线，不是全输入空间形式证明、CDC 证明、综合面积或 STA 频率结论。随机种子固定为 1、17、91；每次请求的最终接收公平性仍依赖上游调度协议。注错运行结果由独立 mutation 报告记录，不混入本报告 PASS 统计。

最终输入约束与源码版本

最后一次回归已采用修正后的输入门控：每拍先令 arvalid=0，驱动其他输入并求值；按实际 DUT aready 是否全零决定是否送出 valid，再求值。若异常实现因 valid 改变再次使 ready 全零，也会在采样前撤销 valid。参考模型的 expected_ready 仍独立计算并比较，绝不使用实际 ready 替代正确性判据。因此即使注错把实际 ready 错误压低，输入仍满足 ready=000 时 valid=0 的约束，且 ready 差异会被检出。

修正后重新构建、运行基础 19 例和补充 4 例，均 PASS，上述总数与必达覆盖不变。

主线程另外执行 Icarus 的 -g2012 -DSYNTHESIS 编译/展开检查并通过；它输出 always_* sensitivity 保守扩展提示。该检查不包含 Icarus 行为仿真，也不是综合面积、综合时序或功能正确性的证明。

Harness	SHA256
cmdbuf_rd_harness.cpp	639bc3d0ebd6dee75e7b3c0060aee39fc50efb75029b8d0dfd7e4528bcbd2e65
cmdbuf_rd_harness_extended.cpp	9d38710a535afc536162e92fe9d6c1a460ff1a102385ea47ed84e5be397d3149

每例覆盖数据索引

下列 JSON 各自包含 checks、coverage、status 和 failure，覆盖门槛定义保存在对应 .case。

Case	完整检查和覆盖计数
01_pipeline	JSON
02_same_xau_ooo	JSON
03_forward_6b	JSON
04_grants	JSON
05_failure_5a	JSON
06_window_block	JSON
07_single_pulse	JSON
08_rr	JSON
09_errors	JSON
10_capacity_reuse	JSON
11_capacity_700	JSON
12_capacity_440	JSON
13_tokens	JSON
14_single_source_throughput	JSON
15_reset_clean	JSON
16_random_seed_1	JSON
17_random_seed_17	JSON
18_random_seed_91	JSON

Case	完整检查和覆盖计数
19_rr_ready_masks	JSON
20_all_bank_halves	JSON
21_repeated_ids	JSON
22_status_after_selection	JSON
23_nonmonotonic_ids	JSON

源码与 case 的完整校验值保存在 verification_sources.json。

cmdbuf_rd 测试敏感性检查与结果

本检查把单个错误植入真实 RTL 副本，再运行与无注错基线完全相同的独立 scoreboard。它是所选关键行为的测试敏感性检查，不代表历史注错目录全部用例已实施。

隔离与可复现

- 基线源码位于 `/root/cmdbuf_rd/rtl`，运行器只读。每次注错前后核对全部 SystemVerilog 文件的 SHA-256。
- 每个变异复制至 `/root/cmdbuf_rd/mutations/Mxx/rtl`，只替换一个且仅出现一次的精确片段。片段缺失、重复或未改变文本均拒绝运行。
- `RTL_ROOT` 指向该变异的项目根目录；`BUILD_DIR`、`RESULT_DIR` 使用变异 ID 和本次运行编号隔离。
- 每批冻结 harness、所选 case 和项目配置，构建显式传入快照的 `HARNESS_FILES`、`CASE_DIR`、`PROJECT_CONFIG`。逐项校验快照及通用环境文件哈希，避免构建期间跨版本。
- 先独立构建无注错副本并跑完选定用例；基线失败立即停止，不计算变异结果。
- 所有命令均通过 `/root/env_base/env/Makefile` 及 `projects/cmdbuf_rd.mk`，复用相同 case 文件和检查器。
- 编译与仿真分别保存日志；基线及每个变异保存结果 JSON，完整汇总位于 `mutations/campaigns/<run-id>/summary.json`。

结果口径

结果	含义
`killed`	构建成功；scoreboard 明确报告失败，或日志明确出现 RTL `Assertion failed in`，运行返回失败且其余用例没有基础设施错误。
`survived`	构建成功；全部所选用例明确通过，当前测试未发现该变异。
`compile_error`	编译失败或编译超时，不计为检测成功。

结果	含义
`runtime_error`	仿真超时、崩溃、缺少结果文件、配置错误或不一致的退出状态，不计为检测成功。
`mutation_error`	精确替换条件不满足或无法创建隔离副本，未有效运行变异。

只有 `killed` 才表示验证确实发现设计错误。`detection_kind=scoreboard` 与 `rtl_assertion` 分开记录；RTL 断言直接终止时可能来不及生成 harness JSON，此时保留具体断言日志作为证据，普通崩溃不算检测成功。检查器失败报告及相应波形保留在变异结果目录。固定接口/容量约束下的长期合法等待不能仅因等待而算作错误。

计划覆盖

已根据真实 RTL 绑定以下 12 个唯一片段，均保留精确变更文本和前后 SHA-256：

ID	故意植入的错误
M01	五项窗口扩大为九项，允许越窗。
M02	双 grant 的 AND 改为 OR。
M03	关闭 6B 成功当拍窗口前馈。
M04	OTB 按 `req` 而非成功握手产生返回 token。
M05	捕获许可额外使用提前一拍的 tap，造成早采样或重复采样。
M06	共享额度从六项放宽到九项，破坏每来源保底。
M07	R 候选忽略当前 `rready`。
M08	仅在已完成项中找来源队头，跳过未完成的老命令。
M09	令奇偶异常覆盖地址越界异常。
M10	当前请求失败时丢弃同拍预选结果，违反 5A。

ID	故意植入的错误
M11	准入不扣当拍退休占用，关闭 10B。
M12	选择时反转奇偶 Bank 状态索引。

它们覆盖独立敏感点，不等价于目录全部 49 组注错。M05 是提前 tap 污染，不是完整 MEM→R 同拍旁路实现的变异。

使用

```
python3 /root/cmdbuf_rd/scripts/run_mutations.py --list
python3 /root/cmdbuf_rd/scripts/run_mutations.py
python3 /root/cmdbuf_rd/scripts/run_mutations.py --only M01,M03
python3 /root/cmdbuf_rd/scripts/run_mutations.py --only M01 --case 06_window_block.case
```

`--manifest` 可提供 JSON 列表（或含 `mutations` 列表的对象）。每项包含 `id`、`file`、`before`、`after`，可选 `purpose`；文件必须是 `rtl/` 下的相对路径，ID 为 `M01` 形式。仅有构建失败与未发现错误等非理想结果时也原样留存，不隐藏失败实验。

最终结果 campaign_03

最终完整批次为 `campaign_03`，无注错快照先通过 **19/19** 用例；随后 **12/12 变异被检测**，`survived=0`、`compile_error=0`、`runtime_error=0`、`mutation_error=0`。每个变异均运行同一组 19 个用例。全部 12 项都有独立 scoreboard 的失败证据，M04、M05 还在其他用例中触发 RTL 断言。

完整证据见 最终汇总 JSON。各变异的精确替换、原文与变更后 SHA-256、编译日志、用例 JSON、失败波形及结果分类均在 `mutations/Mxx/results/campaign_03/` 及汇总中保留。

下表列出按用例执行顺序出现的首个 **scoreboard** 失败。`cycle` 是 harness 从 0 开始的测试周期编号，不直接等同于设计图中的 C1 编号；数值均为十六进制。

变异	首个 scoreboard 报错用例	cycle	首个不一致：实际 / 预期	检测来源
M01	03_forward_6b.case	7	req=0x4 / 0x0	scoreboard
M02	04_grants.case	3	pending_cnt=0x2 / 0x3	scoreboard
M03	03_forward_6b.case	10	req=0x0 / 0x4	scoreboard
M04	05_failure_5a.case	7	rvalid=0x1 / 0x0	scoreboard、RTL assertion
M05	01_pipeline.case	6	rvalid=0x1 / 0x0	scoreboard、RTL assertion
M06	07_single_pulse.case	7	aready=0x7 / 0x6	scoreboard
M07	07_single_pulse.case	7	rvalid=0x1 / 0x0	scoreboard
M08	02_same_xau_ooo.case	8	rvalid=0x1 / 0x0	scoreboard
M09	09_errors.case	10	rresp=0x1 / 0x3	scoreboard
M10	04_grants.case	3	req=0x0 / 0x2	scoreboard

变异	首个 scoreboard 报错用例	cycle	首个不一致：实际 / 预期	检测来源
M11	01_pipeline.case	7	aready=0x6 / 0x7	scoreboard
M12	16_random_seed_1.case	5	req=0x4 / 0x0	scoreboard

M04 按完整执行顺序的首个失败实际来自更早执行的 `04_grants.case`，RTL 报告 `capture ownership mismatch`。该断言中止没有 C++ 周期 JSON，不推测其周期；原始日志保留在 M04 断言日志。上表额外提供首个 scoreboard 用例的明确周期与实际/预期值。M04 有 3 个 scoreboard 检出用例、3 个断言检出用例；M05 分别为 11 个、8 个。

固定版本与工具

本批使用 VM 现有工具：`/usr/local/bin/verilator`，Verilator **5.024**；通过 `/opt/rh/devtoolset-11/enable` 选择 GCC **11.2.1**；运行器 Python **3.6.8**。构建通过现有 `/root/env_base/env/Makefile`，启用 `--Wall --assert`，未关闭警告以让变异构建通过。

最终源码 SHA-256：

文件	SHA-256
`rtl/cmdbuf_rd.sv`	`06e57c016cc74fff564c4d7c2e6e5fc8c9df317f9b172a6f3c7a6df7d74db7e8`
`rtl/cmdbuf_rd_cmd.sv`	`4de7a4022db5d69f3090691d59b3fe42ca673e77020a478ac7584d0378496339`
`rtl/cmdbuf_rd_otb.sv`	`bbca166e8facd72ec22aeb7f1b666df1da9e08fb2efe76f94fa49a25c0ed0068`
冻结的 `cmdbuf_rd_harness.cpp`	`639bc3d0ebd6dee75e7b3c0060aee39fc50efb75029b8d0dfd7e4528bcbd2e65`
冻结的 `cmdbuf_rd.mk`	`89224072b813eaef06d5e81a0b090f2336c54125cf64c5ea777d0f0e830cc4c5`

快照位于 `mutations/campaigns/campaign_03/verification/`。十九个 case 的逐文件哈希以及通用 Makefile、run_cases.py 的哈希均写入最终汇总；结束时 `baseline_source_unchanged=true`、`verification_inputs_unchanged=true`。

最终驱动先令 `arvalid=0` 稳定其他当拍输入，再依据 DUT 实际 `arready` 门控发送意图；采样前保证实际 `arready=000` 时 `arvalid=0`。期望 ready 仍由独立模型计算并比较，因此错误地拉低 ready 会被检查为接收机会损失，不靠违反输入约束检出。

中途批次与检查边界

- campaign_01 保留为 interrupted：RTL 为兼容 Icarus 做了等价 packed 索引改写，批次停止。该批 M04 原始连接替换造成未用信号编译失败，已单列 compile_error；后续修正变异表达式以保留信号使用。M05 当时的日志识别仅认识 %Error，未识别 %Fatal 断言前缀，保守记为 runtime_error；后续修正精确识别，未把普通 abort 归为检测成功。
- campaign_02 保留为 interrupted：独立审查发现驱动器应按实际 DUT ready 门控 valid，故停止并修正 harness；该批部分结果不作为最终结论。
- 最终结论只采用重新完整运行且源码、验证输入固定的 campaign_03。前两批日志和状态保留，未混入最终计数。
- 本次独立静态核对覆盖窗口与当前请求排除、5A/6B、四拍返回标记、MEM 捕获后下一拍 R、来源保序、容量计数、10B 同槽复用及错误优先级；没有提出尚未修复的 RTL 功能问题。上述输入驱动约束问题已由验证适配器修复并重跑。
- 12 项检出证明这组测试对这些具体错误敏感；不等同于全设计证明、全部 49 组历史注错完成或所有边界行为已定义。扩展 harness 的额外 4 个用例未用于本批变异结论。

内容来源　simulation/cmdbuf_rd/docs/verification_report.md

cmdbuf_rd 开发与验证交付

实际开发与验证主机：`192.168.31.116`。工程：`/root/cmdbuf_rd`。日期：2026-10-10。

三个 SystemVerilog 模块已完成：顶层连接、CMDBUF 命令控制、OTB 数据与固定延迟返回。实现了当前已对齐的 9 项、窗口 5、三周期前向流水、5A/6B/10B、四拍 MEM 返回、下一拍才可返回 R、单 XAU 保序、ready 内 RR、错误优先级与来源容量保留。

实际结果

项目	结果
Verilator 静态检查	PASS，启用 --Wall，无警告
公共验证环境结构检查	PASS
独立模型基础回归	19/19 PASS
补充定向回归	4/4 PASS
合计功能周期 / 检查次数	6,733 / 104,987
完整接收→发射→捕获→返回	3,178 条，四阶段计数一致
Icarus 编译与展开	PASS；有工具本身的 always_* 敏感列表扩展提示，未运行 Icarus 功能仿真
最终 RTL 注错	12/12 检出；漏检 0，构建失败 0，运行错误 0，注入错误 0

注错最终批次为 `campaign_03`，使用固定源码及验证环境快照，RTL 前后 SHA256 一致。旧的两个批次中断记录保留在 VM，它们不纳入上述最终统计。注错同时统计独立参考模型和内部 RTL 断言的检测，具体类型见下表；编译失败和环境异常不计为检出。

本次运行的注错

编号	目的	结果	检测途径
M01	错误扩大最老五条窗口，允许第六条绕过阻塞窗口	killed	scoreboard
M02	将双 grant 同时命中错误放宽为任一 grant 命中	killed	scoreboard
M03	关闭 6B 成功当拍窗口前馈	killed	scoreboard
M04	仅发 req 即启动 OTB 返回 token，失败请求也被计入	killed	rtl_assertion, scoreboard
M05	固定四拍捕获错误多使用提前一拍的 tap，导致早采样或重复捕获	killed	rtl_assertion, scoreboard
M06	放宽共享额度，破坏每来源一项容量保底	killed	scoreboard
M07	响应候选忽略当拍 rready	killed	scoreboard
M08	仅在完成项中找来源队头，跳过同来源更老的未完成命令	killed	scoreboard
M09	错误地让奇偶异常覆盖地址越界异常	killed	scoreboard
M10	违反 5A，在当前请求失败时丢弃同拍预选的下一命令	killed	scoreboard

编号	目的	结果	检测途径
M11	关闭 10B 准入的退休前馈，满表时多停一拍	killed	scoreboard
M12	反转命令选择时的奇偶 Bank 状态索引	killed	scoreboard

本次实际运行的是上述 12 个具体 RTL 变异；此前 49 组注错清单仍作为完整规格覆盖来源，并未将清单条目数当作实际注错运行数量。

覆盖与实现选择

定向验证包括窗口内全阻塞而第六条空闲、失败后保持下一条、成功当拍推进窗口、双 grant、固定四拍与单拍 MEM 数据、无同拍 MEM→R 旁路、同源乱序发射但有序返回、ready 内 RR、满表同拍复用、每来源保底容量、四种错误组合、全部 16 个 Bank half、重复与非单调 ID、选中后状态翻转、初始复位及排空后复位。

参考模型按接收序号及队列建立预期，不依赖 RTL 年龄矩阵。驱动器按实际 aready 门控 arvalid，保证在注错情况下也遵守 aready=000 时 arvalid=0 的输入约束；ready 的正确值仍由独立模型检查。

容量实现采用每 XAU 1 项逻辑保底 + 6 项共享，物理 entry 不分区。尚未由系统冻结的细节采用 README 中明确记录的实现选择：异步低有效复位、RR 初始从 0 开始、错误数据透传、请求级不随状态变化撤销等。

文件和复现

- `rtl/`：3 个 RTL 文件。
- `README.md`：接口行为、实现选择和基础运行命令。
- `docs/verification_baseline.md`：每例结果、覆盖计数、基础与扩展回归命令。
- `docs/verification_summary.json`：仿真结构化汇总。
- `docs/mutation_plan.md` 与 `scripts/run_mutations.py`：注错定义和可复现 runner。
- `results/baseline/`、`results/extended/`：逐例日志、JSON 与选定波形。

- `source_manifest.json`：实际交付源码、环境和 case 的 SHA256。
- 随包的 `env_base/`：只包含本项目 adapter/cases/config 和所需公共 runner，未包含其他项目。

在原虚拟机运行：

```
cd /root/cmdbuf_rd
make lint validate
make test
```

扩展回归和注错重跑命令见上述报告，注错重跑请使用新的 run-id。压缩包以 `/root` 为目录基准保存 `cmdbuf_rd/` 和 `env_base/`，可在独立位置解包检查；原 VM 已有完整可运行工程。

验证边界

本次交付为固定配置 RTL 与动态功能验证，不是完整形式证明；未运行综合、STA 或 CDC 验证，不能据此承诺频率和面积。运行中复位涉及未完成 MEM 事务的协同协议仍需系统定义；上游调度公平性未定，不声明无条件最终接收保证。

内容来源　simulation/cmdbuf_rd/docs/design_spec.md

归档说明：本文是实施前设计方案快照。当前 RTL、已采用的实现选择和验证完成情况，以工程 README.md、verification_baseline.md 及 verification_report.md 为准。原文中的“尚未交付”描述保留用于追溯设计阶段状态。

cmdbuf_rd 详细 RTL 设计方案

版本日期：2026-10-10。设计基线：九项 CMDBUF 与九项 OTB、五条待发命令乱序窗口、5A / 6B / 10B 时序、成功发射后固定四拍返回、MEM 捕获后最早下一拍响应，以及每 XAU 保底的容量保护方案。

本方案用于 RTL 编码、模块接口评审和验证建模。已确认的行为作为实现约束；固定 entry、年龄矩阵、相同物理索引等属于本方案的实现选择。复位、精确 RR 顺序等尚未最终定义的边界在文末列出，不将建议冒充已经确认的接口要求。

时序图集 · 注错清单 · 容量隔离说明

1 设计目标和能力边界

cmdbuf_rd 位于上游共享 AR/R 接口与下游 Arbit、MEM Bank 之间。它将命令接收、Bank 可用性判断、仲裁发射、数据捕获和响应返回解耦，允许窗口内较新但目标 Bank 空闲的命令先发射，同时保证同一 XAU 的响应按接收次序返回。

项目	设计基线
上游来源	最多三个 XAU，src 为 0、1、2
AR 接口	一组共享负载，每拍最多接收一条；并非三条独立 AR 总线
总容量	九条未退休命令；CMDBUF 与 OTB 各九项、一一对应
入口容量保护	每个 XAU 逻辑保留一项，其余六项共享；物理槽位统一分配
下游命令带宽	每拍最多尝试发射一条，最多成功一条

项目	设计基线
乱序范围	最老五条未成功发射命令
最短前向路径	C1 接收、C2 判断、C3 请求并获得同拍仲裁结果
MEM 返回	成功发射与 MEM 接收同拍，四拍后返回一组数据及 pc_err，仅有效一拍
R 返回	每拍最多一条；同 XAU 保序，不同 XAU 在合格通道间 RR
最短完整路径	C1 接收、C3 成功、C7 捕获、C8 返回并释放
释放条件	实际向上游返回 R 响应后，同时释放 CMDBUF 与 OTB

本模块能够缓冲 Bank 冲突，并利用五项窗口内的其他空闲 Bank。它不会增大 MEM 的物理接收带宽，也不保证所有状态下持续有命令可发。最老五条全部被阻塞时，第六条即使 Bank 空闲也继续等待。

容量保留保证零占用 XAU 至少有首条命令的接收名额。用户已确认上游可以切换 src，但公平性未定；模块不能强迫上游选择某个来源，也不能无条件保证其请求最终被调度。

2 顶层接口

以下方向均相对于 cmdbuf_rd。为避免同名，AR/R 的 port_id 在本文分别命名为 ar_port_id 和 r_port_id；最终端口命名可按工程规范调整。

2 1 时钟和复位

信号	方向	位宽	说明
clk	输入	1	统一时钟；当前方案按单时钟域设计

信号	方向	位宽	说明
reset	输入	1	占位名称；极性、同步或异步及运行中复位协议待确认

若上游、Arbit 或 MEM 实际跨时钟域，需在模块边界增加单独的跨域协议，不能直接沿用本文固定四拍关系。

2 2 上游 AR 通道

信号	方向	位宽	含义
arvalid	输入	1	当前共享 AR 负载有效
arready	输出	3	每个 XAU 的接收资格；三位允许不同，也不要求 one-hot
araddr	输入	18	命令地址
src	输入	2	来源 XAU，合法值 0～2
arid	输入	8	事务标签，保存后透传到 rid
ar_port_id	输入	5	保存后透传到 r_port_id
arresp	输入	2	00 正常、11 地址越界；其他值未定义

```
ar_fire = arvalid && (src < 3) && arready[src]
```

实现时先判断 src 合法性，再选择 ready 位，避免对 src=3 做越界索引。上游保证 arready 全零时 arvalid=0；这不等价于“任一 ready 为 1 就可接收当前 src”。

arid 不参与年龄排序，不要求数值随命令先后递增。本实现不依赖 ID 唯一性；响应与 entry 的归属使用内部标识和接收年龄。

2 3 上游 R 通道

信号	方向	位宽	含义
rvalid	输出	3	通过哪一位为 1 指示接收 XAU；至多一位为 1
rready	输入	3	三个 XAU 当前接收响应的能力
rdata	输出	128	一组共享数据
rid	输出	8	原 arid
r_port_id	输出	5	原 ar_port_id
raddr	输出	3	原 araddr[2:0]
rresp	输出	2	00 正常、11 地址越界、01 奇偶校验异常

```
(rvalid & ~rready) == 0
onehot0(rvalid)
retire_fire = |(rvalid & rready)
```

本接口约定只在 ready=1 的通道上产生 valid，因此没有“先拉高 valid，再锁住共享负载等待 ready”的状态。所有 R 字段来自同一获选命令。

2 4 Arbit 接口

信号	方向	位宽	含义
req	输出	8	Bank 请求，one-hot 或全零
arb_addr	输出	12	请求级保存地址的 araddr[17:6]
pending_cnt	输出	4	全部已接收但未成功发射的命令数，范围 0～9
ar_rd_gnt	输入	8	下游仲裁响应

pending_cnt 表示全局待发数，不是窗口内命令数，也不是总占用数。输出使用当前寄存状态，不把当拍 grant 反馈形成的下一状态计数直接送回 Arbit。

2 5 MEM 接口

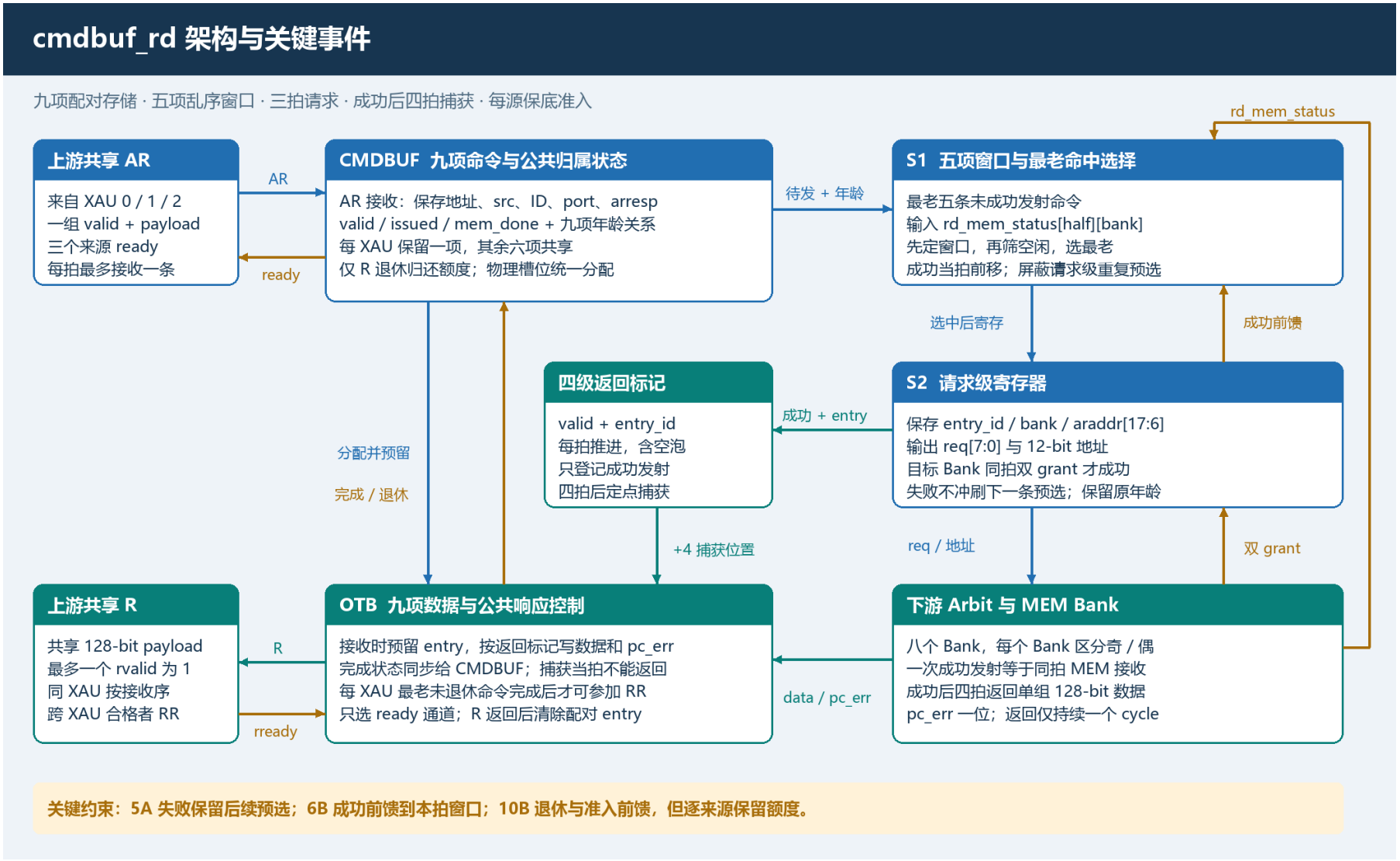
信号	方向	位宽	含义
rd_mem_status	输入	[1:0][7:0]	第一维奇偶，第二维 Bank；0 空闲、1 忙
mem_rdata	输入	128	合并后的单组返回数据
pc_err	输入	1	当前有效返回的奇偶校验异常
rw_rd_gnt	输入	8	目标 Bank 的读仲裁许可

```
bank_id   = saved_araddr[5:3]       // Bank 0～7
bank_half = saved_araddr[6]         // 0 偶、1 奇
bank_idle = !rd_mem_status[bank_half][bank_id]
arb_addr  = issue_addr_q            // 保存的 araddr[17:6]
```

arb_addr[0] 已携带原地址 bit 6，无须再添加 bank_half 外部端口。没有独立 MEM 返回 valid 的已定义端口；本模块依据成功发射事件固定延迟四拍，产生内部捕获有效信号。pc_err 不能充当返回 valid。

当前协议是一条 AR 对应一次 128-bit 返回，不额外引入 burst、rlast、AXI 重排规则或未约定的地址对齐限制。

3 总体架构　cmdbuf_rd 架构



cmdbuf_rd 架构

顶层仍由 CMDBUF 和 OTB 两个主要部分组成。控制职责建议如下：

功能	归属建议	作用
AR 接收与配对分配	CMDBUF 侧公共控制	计算每源 ready，原子分配 CMDBUF/OTB
命令元数据与年龄	CMDBUF	保存字段、维护先后关系
五项窗口和 Bank 判断	CMDBUF	从合格窗口选择最老命令
请求级寄存器	CMDBUF	下一拍输出 req/address，取得同拍 grant
成功发射标记流水	OTB 返回控制	将内部 entry 标识延迟四拍
数据与 pc_err 存储	OTB	无背压捕获 MEM 单拍返回
每源队头、RR、共享 R mux	公共响应控制	按同源接收顺序返回
完成和退休广播	公共控制	MEM 完成通知 CMDBUF；R 返回释放两侧

AR 接收时立即为 OTB 保留位置。请求级发出命令时，同步将命令归属传给 OTB；只有成功发射事件才能启动返回标记流水。MEM 数据回来时无需再次争抢容量。

4 存储组织与内部状态

4 1 固定九项 entry

本方案采用固定 entry 编号 0～8，内部 entry_id 为 4 bit，9～15 为无效编码。CMDBUF 与 OTB 使用相同 entry_id，使分配、返回和释放容易对应。

相同物理编号是实现选择；外部要求是一一对应。若实现为不同物理编号，必须增加可靠映射并保持本文的行为。

命令不会因成功发射而搬移到另一物理槽，槽位只在 R 返回后成为空闲。乱序、新老关系由年龄结构表达，不由槽位编号表达。

4 2 每项字段

字段	位宽	归属	更新时机
valid	1	公共归属状态	AR 接收置位，R 退休清除
issued	1	CMDBUF	分配置零，成功发射置位
mem_done	1	公共完成状态	分配置零，MEM 捕获置位
cmd_addr	18	CMDBUF	AR 接收保存
cmd_src	2	CMDBUF	AR 接收保存
cmd_id	8	CMDBUF	AR 接收保存
cmd_port_id	5	CMDBUF	AR 接收保存
cmd_arresp	2	CMDBUF	AR 接收保存
data	128	OTB	MEM 捕获保存
saved_pc_err	1	OTB	与 data 同拍保存

valid 与 mem_done 可以作为两部分共享的权威状态。CMDBUF 直接使用 mem_done 感知返回，OTB 以相同状态表示数据有效，避免维护两个独立完成位而发生失配。若为了模块边界而镜像状态，两侧必须在同一事件下更新并检查一致性。

本方案建议将 OTB 数据用寄存器阵列实现。九项共 1152 个数据 bit，能够为不同 entry 同拍执行捕获和清除；不能未经端口分析就替换成单写口 RAM。

4 3 派生状态

状态	条件	是否占容量	是否计入待发

正在请求级尝试的命令仍属于 WAIT，直至取得成功 grant；额外的 issue_valid_q/issue_entry_q 仅用于标识流水占用、屏蔽重复预选。失败不需要释放、重新分配或更改年龄。

```
FREE --AR 接收--> WAIT --成功发射--> INFLIGHT --MEM 捕获--> DONE --R 返回--> FREE
                    |
                    +--仲裁失败--> WAIT，年龄不变
```

4 4 全局寄存器

状态	建议宽度	用途
older_set	9 × 9	每项有哪些更老的有效命令
issue_valid_q	1	请求级有效
issue_entry_q	4	请求级 entry
issue_bank_q	3	请求级目标 Bank
issue_addr_q	12	请求级下游地址
ret_valid_q	4	固定四拍返回标记
ret_entry_q	4 × 4	每级返回标记对应 entry
occ_q	3 × 4	各 XAU 已接收、未退休总占用
used_cnt_q	4	总占用，0～9；可由 occ 求和派生
pending_cnt_q	4	总待发数，0～9
rr_ptr_q	2	RR 扫描起点，合法值 0～2

5 AR 准入和容量隔离

5 1 逻辑保留与共享额度

设计采用每 XAU 逻辑保留一项、其余六项共享。只有一个 XAU 活动时最多占七项，另两项继续为其他来源保留，不因其暂时没有请求而借出。

occ[x] 统计该 XAU 全部 valid 命令。成功发射以及 MEM 返回均不会减少它；只有 R 退休才减少。

```
sum(max(occ[x], 1)) <= 9

等价于：sum(max(occ[x] - 1, 0)) <= 6
```

只限制每 XAU 最多七项不够，因为 (4,5,0) 仍会占满九项并挡住第三路。上述不变量同时保护每个零占用来源的首条接收名额。

5 2 考虑当拍退休后的 ready

```
retire[x] = rvalid[x] && rready[x]
occ_after[x] = occ_q[x] - retire[x]
free_after = 9 - sum(occ_after)
shared_after = sum(max(occ_after[x] - 1, 0))

arready[x] = (free_after > 0)
          && ((occ_after[x] == 0) || (shared_after < 6))

accept[x] = ar_fire && (src == x)
occ_next[x] = occ_after[x] + accept[x]
```

减法、比较和加法使用足够宽的无符号中间值；occ=0 时的 max(occ-1,0) 必须先判断，不能发生下溢。reset 期间的 ready 输出另外受复位策略控制。

原占用 (XAU0,XAU1,XAU2)	当拍退休	arready[2:0]
(7,0,0)	无	110
(7,1,0)	无	100

原占用(XAU0,XAU1,XAU2)	当拍退休	arready[2:0]
(7,1,1)	无	000
(4,4,0)	无	100
(7,1,1)	XAU1 唯一一条	010
(7,1,1)	XAU0 一条	111

若某来源 occ_after[x]=0，保留不变量保证至少还有一个空槽；它接收第一条不增加 max(occ_after[x],1)，因此不依赖其他来源退休即可准入。

5 3 物理 entry 分配

```
free_mask = ~valid_q | retire_mask
alloc_valid = ar_fire
alloc_entry = select_one(free_mask)
```

可以用固定最低空闲编号选择器，分配顺序不参与年龄优先级。不存在两个 AR 同拍竞争分配的问题，因为总线每拍最多一条。

接收沿原子完成：置 valid、保存所有 AR 字段、清 issued/mem_done、初始化 OTB 状态、把新命令登记为最年轻。新命令只能从下一周期开始参与窗口判断。

6 接收年龄管理

本方案采用九项年龄矩阵，避免有限宽度时间戳溢出后比较新老的问题。

定义 older_set[i][j]=1 表示“有效 entry j 比有效 entry i 更老”。对角线恒为零；任意两个不同有效 entry 的先后关系唯一。

```
oldest(M)[i] = M[i] && !(|(older_set[i] & M))
```

其中 M 是待比较集合。例如 M 为窗口内可发集合时，返回最老可发命令；M 为某 XAU 的所有 valid 项时，返回其最老未退休命令。

6 1 更新规则

只在分配或退休改变年龄关系；发射、失败、MEM 完成不改变年龄。

在先求出 valid_next 后，对每对 i、j 使用下列优先级：

```
if (!valid_next[i] || !valid_next[j] || i == j)
    older_next[i][j] = 0;
else if (alloc_valid && i == alloc_entry)
    older_next[i][j] = 1;  // 所有其他存活项都比新 i 老
else if (alloc_valid && j == alloc_entry)
    older_next[i][j] = 0;  // 新 j 不可能比旧 i 老
else
    older_next[i][j] = older_q[i][j];
```

退休后无复用时，该项所在行列清零；同拍退休并复用同一槽时，新分配规则重建该行列，不能保留旧命令年龄。

九项矩阵只有 81 个 bit，易于实现与检查。若实现优化为三角矩阵，行为与比较方向应保持不变。

7 三拍前向流水

周期	阶段	工作
C1	S0 接收	AR 握手，配对分配，保存命令
C2	S1 判断	构建五项窗口，匹配 Bank 空闲，选最老合格项；拍末寄存
C3	S2 请求	输出 req/address，当拍获得两种 grant，拍末提交成功或失败

S0、S1、S2 针对不同命令并行运行。C3 可以同时接收 C、判断 B、请求 A。不能将三阶段做成等待前一命令完成后才处理下一条的串行状态机。

S1 没有候选则向 S2 注入空泡。S2 的每次尝试在该周期结束，不因失败而保留原请求级内容；下一周期请求级接收的是本周期 S1 的结果。

8 五项窗口和最老匹配选择

8 1 候选集合

```
pending_mask = valid_q & ~issued_q
current_issue_mask = issue_valid_q ? onehot(issue_entry_q) : 0
issue_success = |(req & ar_rd_gnt & rw_rd_gnt)
success_mask = issue_success ? current_issue_mask : 0


rank_pool = pending_mask & ~success_mask


older_count[i] = popcount(older_set[i] & rank_pool)
window_mask[i] = rank_pool[i] && (older_count[i] < 5)


bank_idle_mask[i] = !rd_mem_status[cmd_addr[i][6]][cmd_addr[i][5:3]]
candidate_mask = window_mask & bank_idle_mask & ~current_issue_mask
select_onehot = oldest(candidate_mask)
```

顺序必须是先决定最老五条，再筛 Bank 空闲。若先在全部九项中过滤空闲 Bank，会把窗口外命令错误引入候选。

所有 entry 的 Bank 状态可以并行读取。窗口 rank 最大为八，计数比较使用四位即可。若候选存在，年龄关系保证 oldest 返回唯一命令。

8 2 6B 成功当拍前移

当前请求级 A 成功时，success_mask 在本拍 S1 计算中将 A 排除。原第六条 F 可当拍进入窗口，并在下一拍发出请求。

```
N：请求 A 成功；B～E 忙；F 空闲 → S1 当拍选 F
N+1：请求 F
```

如果等 issued 寄存器下一拍更新后才前移，会多一个气泡，与所选 6B 不符。

8 3 失败时的窗口位置

当前请求级 A 失败时，A 仍留在 rank_pool 和最老五条窗口中，只在最后候选掩码中被屏蔽，避免同拍重复预选。

不能在构窗前无条件移除 current_issue_mask，否则 A 失败也会让 F 错误进入窗口。

8 4 五条全部阻塞

A ~ F 从老到新，A ~ E 对应 Bank 均忙、F 空闲且当前请求级为空时，candidate_mask=0。

rd_mem_status 若持续不变，窗口不会自动滑动，F 不发射。

这一等待是规格行为。每 XAU 容量保留只能让其他来源的命令进入 CMDBUF，不能授权窗口外命令绕过发射规则。

9 请求级和 5A 失败恢复

9 1 请求级寄存器

在每个周期结束时：

```
issue_valid_q <= select_valid
if (select_valid):
    issue_entry_q <= select_entry
    issue_bank_q  <= cmd_addr[select_entry][5:3]
    issue_addr_q  <= cmd_addr[select_entry][17:6]
```

下一周期输出：

```
req = issue_valid_q ? (8'b1 << issue_bank_q) : 8'b0
arb_addr = issue_addr_q
issue_success = issue_valid_q
             && ar_rd_gnt[issue_bank_q]
             && rw_rd_gnt[issue_bank_q]
```

req 必须来自请求级寄存器，不能由本拍 grant 决定是否产生，否则会引入仲裁组合环。无请求时的 arb_addr 可输出寄存值或零；本方案建议对外置零便于观察，此行为不是已确认功能要求。

9 2 5A 的恢复过程

周期	S1 判断	S2 请求
C2	A	空
C3	B	A 失败
C4	A	B
C5	后续命令	A 重试

A 失败不冲刷 B，也不在失败当拍重新选择 A 替换 B。A 的 valid 保留、issued 仍为零、年龄不变；下一拍自动恢复候选资格。

如果只有 A 一条命令，C3 失败后，C4 重判、C5 重试。这一重试间隔来自已经选择的流水行为。

9 3 请求拍 Bank 状态变化

已确认的成功判据始终是目标 Bank 的同拍双 grant。判断拍空闲、请求拍转忙时，是否额外撤销请求尚未独立对齐。

本方案建议锁定 S1 的选择，在 S2 正常输出请求，由双 grant 决定是否接受，避免加入第二套隐含选择规则。该建议列为待确认实现边界。

9 4 同步给 OTB 的信息

CMDBUF 在请求级同时向 OTB 提供 attempt_valid、entry_id 及 issue_success。最小功能信息是命令归属与成功提交；完整地址和元数据已由接收时保存，不必重复存入 OTB。

attempt_valid 便于观察请求尝试；只有 issue_success 才允许 OTB 产生返回标记。失败不能造成 OTB 完成计时、重复分配或提前清除。

10 固定四拍返回标记

下游成功接收后不能反压返回，因此使用四级 valid 加 entry_id 标记流水，按时钟无条件推进。

```
每个时钟沿：
    ret_valid_q[0] <= issue_success
  成功时 ret_entry_q[0] <= issue_entry_q


  对 k=1..3：
      ret_valid_q[k] <= ret_valid_q[k-1]
    前一级有效时 ret_entry_q[k] <= ret_entry_q[k-1]


  若本沿前的 ret_valid_q[3] 为 1：
      capture_entry = 本沿前的 ret_entry_q[3]
      otb_data[capture_entry] <= mem_rdata
      otb_pc_err[capture_entry] <= pc_err
      mem_done[capture_entry] <= 1
```

在 SystemVerilog 中用同一沿的非阻塞赋值读取旧末级即可。不要将 capture_valid 再额外寄存一拍，也不要在捕获时使用刚移入末级的下一状态。

时钟沿	A 标记和数据的动作
C3 末	A 成功，写入第 0 级
C4 末	A 移入第 1 级
C5 末	A 移入第 2 级
C6 末	A 移入第 3 级
C7 末	读取旧第 3 级，捕获 A 数据和 pc_err
C8 周期	A 的已寄存完成状态可参与 R 选择

空泡、仲裁失败写入无效标记；已有标记继续前进。AR 满、没有新请求、rready 全零均不能暂停该流水。

外部无返回事务标识，本方案依赖“每个成功命令恰好四拍后返回，且返回合并总线保持该对应关系”的接口约束。若下游将来改为可变延迟，必须增加可匹配的返回标签或改变协议，当前标记流水不能继续直接使用。

11 MEM 捕获和完成通知

捕获当拍将完整 128-bit 数据和 pc_err 写到标记指定的 OTB entry，并使该 entry 的 mem_done 置位。CMDBUF 使用同一完成状态或接收同拍完成通知。

捕获条件只由 ret_valid_q[3] 决定。数据全零、pc_err=0 仍是合法返回；不能以数据非零或错误有效代替捕获有效。

捕获时 entry 必须 valid、issued 且尚未 mem_done。它仍占用容量，后续 rready 不允许响应时，数据和错误保持直到退休。

当前 MEM 捕获不能组合旁路进入本拍 R 候选。R 只能读取本周期开始前已经寄存的 mem_done 和 OTB 数据，从而保证最早下一 cycle 返回。

12 每 XAU 保序及 RR 响应

12 1 每源队头

```
src_mask[x][i] = valid_q[i] && (cmd_src[i] == x)
head_onehot[x] = oldest(src_mask[x])
head_valid[x] = |head_onehot[x]
head_done[x] = mem_done[head_entry[x]]，仅在 head_valid 时读取


r_candidate[x] = head_valid[x] && head_done[x] && rready[x]
```

先找该 XAU 最老的未退休命令，再检查它是否完成。不能在该 XAU 的所有完成项里找最老，否则会跳过尚未完成的真正队头。

不同 arid 的同源请求也必须遵循同一接收序列。不同 XAU 不施加全局返回顺序：XAU0 的队头未完成时，XAU1 的合格队头仍可响应。

12 2 RR 实现建议

当前已确认的是在合格通道间 RR。为了给出可编码方案，建议 rr_ptr 表示扫描起点，按 0→1→2 循环，从起点扫描三个通道，选第一个 r_candidate=1 的通道。

```
winner = rr_first(r_candidate, rr_ptr_q)
rvalid = winner_valid ? onehot(winner) : 0
```

```
若 retire_fire: rr_ptr_next = (winner == 2) ? 0 : winner + 1
否则: rr_ptr_next = rr_ptr_q
```

图 08 使用上述方向和空闲保持；初始起点及精确推进策略尚未独立确认。本方案建议初始指针为 0，但不能把这一建议当作既定系统复位协议。

若三个通道持续有完成队头且持续 ready，上述 RR 方案会循环服务三个通道。它不为非 ready 或队头未完成的通道预留 R 总线。

12 3 共享负载选择

```
response_entry = head_entry[winner]
rdata      = otb_data[response_entry]
rid        = cmd_id[response_entry]
r_port_id  = cmd_port_id[response_entry]
raddr      = cmd_addr[response_entry][2:0]
rresp      = error_encode(cmd_arresp[response_entry], otb_pc_err[response_entry])
```

winner 无效时建议将共享负载置零，便于波形和降低无意义翻转；协议检查只在对应 rvalid 有效时比较负载。

不能用实时 AR 字段或实时 MEM 数据构造 R 负载。R 被选中时对应 rready 已经为 1，因此该周期结束沿同时完成传输和退休。

13 错误处理

保存的 arresp	保存的 pc_err	rresp
00	0	00
00	1	01
11	0	11
11	1	11

地址越界优先于奇偶校验异常。arresp=11 的命令仍完整经历分配、窗口选择、请求、MEM 返回和 R 返回，不做本地立即完成。

```
if saved_arresp == 2'b11: rresp = 2'b11
else if saved_pc_err:    rresp = 2'b01
else:                    rresp = 2'b00
```

该编码逻辑仅对已定义的 arresp=00/11 给出协议承诺。错误响应的数据内容尚未独立约定；本方案建议仍输出捕获值，同时以 rresp 指示数据状态，最终需确认。

14 R 退休和 10B 同拍复用

R 成功传输产生唯一 retire_entry，同时清除旧命令在 CMDBUF、OTB 的有效归属、完成状态和年龄关系，并扣除所属 XAU 的占用。

本方案建议分配及退休时将 OTB data/pc_err 清零。功能正确性至少要求旧数据失效且不能暴露给新事务；若为功耗或 RAM 实现改为只清有效位，是否符合“清除”的最终要求需确认。

14 1 满表复用

free_mask 包含当拍 retire_mask。因此即使沿前九项均 valid，仍能选中本拍即将退休的槽位分配新请求。每来源额度也必须先扣除当拍退休再计算。

例如 (7,1,1) 中 XAU1 唯一命令退休，本拍只能对 XAU1 给出 ready=1；若退休的是 XAU0 的一条，则释放的是共享额度，三路均可具备准入资格。

14 2 同一槽位更新优先级

R 负载由沿前旧状态输出；时钟沿上同槽新分配优先于旧退休清除：

```
if reset_active:
  清控制状态
else if alloc_valid && alloc_entry == i:
    valid=1; issued=0; mem_done=0
  保存新 AR 元数据；清新 OTB 数据状态；重新登记最年轻年龄
else if retire_fire && retire_entry == i:
    valid=0; issued=0; mem_done=0
  清旧归属及对应存储
else:
  若成功发射命中 i，issued=1
```

若 MEM 捕获命中 i，保存数据和 pc_err，mem_done=1

这是每项的逻辑优先级，不代表将所有数组写成多个冲突的 always_ff。每个寄存器应有唯一写入过程；并行事件分别作用于不同 entry，不能用顶层全局 if/else 误删其他事件。

14 3 合法并发关系

同拍事件	可否同 entry	说明
R 退休与 AR 分配	可以	10B，输出旧负载后建立新归属
MEM 捕获与 R 退休	不可以	本拍捕获项此前尚未完成，不能本拍返回
成功发射与 R 退休	不可以	待发与已完成状态互斥
成功发射与 MEM 捕获	不可以	同一命令相隔四拍；出现同项通常是重复发射错误
AR 分配与旧返回标记命中	不可以	未捕获命令不能先退休，其槽位不能被重分配
上述事件作用于不同 entry	可以	可同时接收、成功发射、捕获、返回

正常协议下，旧命令退休时已不存在其未到达的返回标记，因此 entry 重用无需额外 generation tag。复位打断未完成事务是另一边界，见第 21 节。

15 计数及状态一致性

```
used_next = used_q + ar_fire - retire_fire
pending_next = pending_q + ar_fire - issue_success
occ_next[x] = occ_q[x] + accept[x] - retire[x]


sum(occ_q) == used_q == popcount(valid_q)
pending_q == popcount(valid_q & ~issued_q)
0 <= pending_q <= used_q <= 9
sum(max(occ_q[x],1)) <= 9
```

预选、失败、MEM 捕获不改变 pending_cnt。R 退休的命令早已 issued，退休本身也不再减少 pending_cnt。

6B 可以在本拍窗口计算中提前排除成功项，但 pending_cnt 对外仍表示本周期寄存状态，并在本周期末更新。这样不把下游 grant 通过计数绕回下游仲裁逻辑。

实现可以选择寄存计数或由位向量计算，但保留额度计算、对外指示和验证采样必须使用一致的周期定义。

16 周期内组合依赖

建议按以下依赖形成组合逻辑，并统一在周期结束沿提交：

```
当前命令/OTB/年龄状态 + rready
  → 各 XAU 队头和合格候选 → RR → R 负载/retire
  → 扣除 retire 后的占用 → 每来源 arready → AR 接收/分配

当前请求级寄存器
    → req/address → 下游两组 grant → issue_success
  → 成功前馈后的窗口 → Bank 匹配/最老选择 → 下一周期请求级

返回标记旧末级 + 当前 MEM 返回
  → 本沿捕获 → 下一周期可见的 mem_done/data
```

当前 R 选择不依赖新 AR，当前 req 不依赖本拍选择结果，当前 R 候选不依赖本拍 capture_next。上述隔离是保持三拍路径、禁止同拍 MEM 旁路及避免本模块内部组合环的关键。

17 性能与时序实现考虑

17 1 理想吞吐

当上游每拍供给、窗口中持续存在可发项、每次双 grant 成功、R 持续可返回时，模块可每拍接收一条、成功发射一条、捕获一条和返回一条。

单 XAU 最多七项仍能支撑这一理想吞吐：C1～C7 接收七条，第一条 C3 成功、C7 捕获、C8 返回；C8 同时接收第八条，随后每拍释放与接收相抵。这里必须对容量额度做当拍退休前馈，即使物理池中还剩另两路保留空槽也一样。

若没有 10B，单来源达到七项时会额外停一拍。若增加额外 R 流水级，或 MEM 延迟不再是四拍，则这一七项支撑一条每拍的结论也需重新评估。

17 2 关键组合路径

路径	关注点
grant → 成功前馈 → 窗口 → 最老选择 → 请求级寄存器	6B 明确要求这条路径在一拍内完成
mem_done/年龄 → 每源队头 → RR → 共享 128-bit mux	影响 R 返回周期
rready → RR/retire → 每源额度 → arready	10B 和容量保留引入的前馈路径
AR 握手 → 元数据写入及年龄更新	不能越过 S0 寄存直接影响本拍判断

九项年龄比较、Bank 空闲解码和每源队头可并行计算。可预先计算“当前请求成功/不成功”两套窗口，再由成功信号选择，以缩短 late grant 路径；必须证明与统一窗口公式等价。

为了频率随意再插一拍会改变 6B、三拍请求或最早 R 返回时序，不能在保持原规格的名义下增加延迟。目标频率、工艺、综合约束尚未给出，本方案不承诺具体 MHz。

17 3 存储量估算

按本文字段直接实现，未计组合逻辑及实现相关冗余：

项目	bit 数
命令元数据与 valid/issued/mem_done	9 × 38 = 342
OTB 数据及 pc_err	9 × 129 = 1161
年龄矩阵	81
请求级	20
返回标记流水	20
三路占用、总占用、待发数、RR 指针	22
合计	约 1646

这是寄存状态规模估算，不是综合面积结果。共享状态无需在两个模块中重复实现；是否复制、编码或优化由综合和模块边界决定。

18 典型时序

18 1 连续无阻塞

周期	AR 接收	S1 选中	S2 请求成功	MEM 捕获	R 返回
C1	A	—	—	—	—
C2	B	A	—	—	—
C3	C	B	A	—	—
C4	—	C	B	—	—
C5	—	—	C	—	—
C6	—	—	—	—	—
C7	—	—	—	A	—
C8	—	—	—	B	A
C9	—	—	—	C	B
C10	—	—	—	—	C

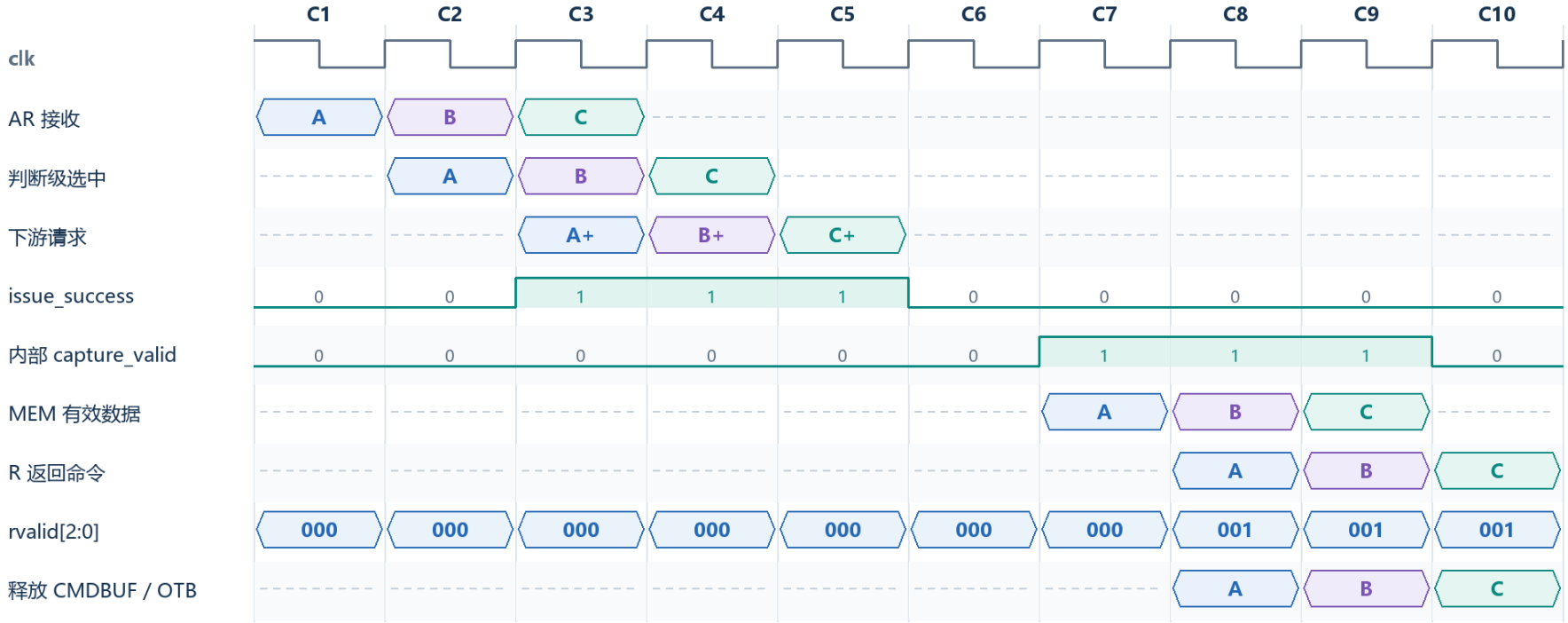
连续流水时序

01　连续正常流水

已确认 无旁路

每列一个周期，右边界上升沿提交；+ 成功，× 失败，虚线表示无对应有效数据。

A、B、C 均来自 XAU 0；仲裁成功；rready[0] 始终为 1



C3 同时接收 C、判断 B、请求 A。A 在 C3 发射成功、C7 捕获返回、C8 响应成功后释放。

已确认：MEM 返回当拍写 OTB，最早下一拍参与 R 仲裁；仍须满足源内保序、rready 和 RR。

连续流水时序

18 2 已对齐场景索引

图	核心行为
01 连续流水	三拍请求、固定四拍捕获、下一拍可返回
02 同 XAU 保序	后接收先完成的命令等待前序返回
03 窗口前移	第六条通过成功前馈进入窗口
04 双 grant 与空泡	失败没有返回标记
05 选择 5A	A 失败保留已选 B
06 选择 6B	成功当拍选入后续项
07 单拍返回	R 不 ready 时仍捕获并保持
08 RR 示例	只选择 ready 的完成队头；精确环方向见待定项
09 字段与错误	地址切片、字段透传、异常优先级
10 选择 10B	满表同拍退休和接收
11 窗口全阻塞	不因第六条可用而越窗发射
12 容量保留	一路占七项后其他来源仍可准入
13 保底复用	释放唯一项时仅恢复该来源资格

19 建议 RTL 划分

文件或逻辑块建议	主要内容
cmdbuf_rd.sv	顶层端口、公共事件、模块连接

文件或逻辑块建议	主要内容
cmdbuf_rd_cmd.sv	九项元数据、状态、年龄、窗口及请求级
cmdbuf_rd_otb.sv	九项数据、pc_err、固定四拍返回标记与捕获
cmdbuf_rd_alloc.sv	每来源占用、保留额度、free mask 与分配
cmdbuf_rd_rsp.sv	每来源队头、RR、共享 R mux 与退休

文件划分是建议，允许将 alloc/rsp 合并到顶层或 CMDBUF 控制中。核心是状态单一归属、每个寄存器唯一写入者，以及分配/完成/退休广播的一致性。

常量建议统一定义：XAU_NUM=3、ENTRY_NUM=9、WINDOW_SIZE=5、BANK_NUM=8、ADDR_WIDTH=18、DATA_WIDTH=128、ID_WIDTH=8、PORT_ID_WIDTH=5、MEM_LATENCY=4。参数化不代表任意组合均已验证；首先固定这一基线实现。

20 验证和关键检查

20 1 基准模型

每次合法 AR 握手分配独立测试序号 accept_seq，保存所有输入字段。为每个 XAU 建接收队列，并维护全局占用、待发集合、成功发射四拍日历以及完成记录。

参考模型按本方案窗口规则选下一拍请求，不使用 arid 数值决定新老。MEM 在对应日历拍提供有效数据；无效拍可以改变总线，暴露晚采样和实时总线误用。

R 只从此前已完成的各 XAU 队头选择。检查完整负载属于同一命令；错误数据值尚未确定时，错误子例优先核对状态与归属，正常子例逐位比较 128-bit 数据。

20 2 安全性检查

以下是断言意图，实际 SVA 需绑定 RTL 信号及采样语义：

- req 与 rvalid 均 onehot0；rvalid 不得指向当拍 rready=0 的通道。
- ar_fire 必须命中合法 src 的 ready；一次 AR 最多分配一个配对 entry。

- mem_done 必须蕴含 valid 和 issued；issued 不得在 grant 失败时置位。
- 每条命令最多成功发射一次、MEM 捕获一次、R 返回一次。
- 捕获 entry 必须有效、已成功发射且未完成；同条命令本拍捕获时不能本拍退休。
- 每次成功发射恰好在四拍后对应一次捕获，失败不得生成返回标记。
- 预选命令必须属于成功前馈后的最老五条集合，目标 half 空闲，且不是当前请求级项。
- 候选非空时，选中者不存在同集合内更老命令；候选为空则不产生选择有效。
- 同 XAU 响应顺序与实际 AR 握手顺序一致；所有元数据和数据归属一致。
- valid、计数、每源占用和年龄结构一致，保留容量不变量一直保持。
- 同拍复用输出旧命令，新命令在下一状态有效、最年轻、未发射、未完成。

20 3 吞吐检查

5A、6B、10B 需要检查精确相邻周期行为，不能只检查最终数据正确。额外空泡也可能是规格偏离。

重点覆盖单 XAU 达七项后的持续接收、全表九项的跨源复用、失败时保留 B、成功当拍引入 F、同时接收/发射/捕获/响应四事件。

20 4 进展条件

无条件安全性检查与有环境前提的活性检查分开。rd_mem_status 持续阻塞、上游公平性未定、rready 永久为零或仲裁持续失败时，不要求每条命令最终完成。

现有 49 组注错清单 可作为验证覆盖来源。容量准入模型已对 190 个合法占用状态及 1716 个接收状态转换进行穷举检查；这是抽象容量模型结果，不是完整 RTL 形式验证或时序收敛结果。

21 复位与异常边界

复位建议将 valid、issued、mem_done、请求级 valid、四拍返回 valid、年龄矩阵、计数和 RR 控制清零。复位期间 req/rvalid/arready 建议全零，具体极性和生效方式待系统确认。

数据阵列可在无效状态下不做复位，由分配和退休清零；外部负载在无 valid 时按实现建议置零。若要求所有存储 bit 复位为零，可调整，但会影响复位网络负担。

运行中复位不能只清本地标记而默认下游旧返回自动消失。需要系统明确同步冲刷 MEM，或停发并排空旧事务等协议，才能定义何时恢复接收。当前方案不自行选定恢复流程。

项目	已知行为	本方案建议或仍待确定内容
容量隔离	不允许 XAU 独占导致其他来源没有 entry	已纳入每源一项保底、六项共享的实现方案
复位	尚无完整协议	极性、同步方式、初始 RR 和未完成返回处置待确认
非法输入	src 0～2、arresp 00/11 有定义	src=3、arresp=01/10 的恢复行为未定义
重复 ID	arid 为透传标签	本实现能按内部归属区分；系统是否另有限制待说明
数据清除	退休后旧归属和数据不能再次有效	建议分配及退休物理清零；是否必须逐 bit 清零待确认
错误响应数据	rresp 优先级已确认	建议仍输出捕获值，最终数据语义待确认
RR 细节	合格通道间 RR	建议 0→1→2、服务后推进、空闲保持，精确规则待确认
Bank 状态变化	双 grant 决定最终成功	建议请求级不二次撤销，是否允许撤销待确认
无效周期负载	无有效请求或响应时不作功能比较	建议输出零，尚无强制值约定
上游公平性	已确认可切换 src	公平性未定，不建立无条件最终接收保证

22 实施交付边界

本方案完成了接口、状态、选择算法、返回匹配、容量准入、并发更新优先级和验证判据的设计定义。可先据此搭建 RTL 主体与参考模型；将第 21 节中会影响外部行为的选择记录为可追踪配置，最终冻结前按系统要求确定。

现有时序图是规格示意，容量检查是独立抽象模型。尚未交付可综合 RTL、运行完整仿真、综合或静态时序分析，不能据此宣称目标频率、面积或全部验证已通过。